

From Reinforcement Learning to Agentic AI

Interaction, tools, memory, and scientific discovery

FYS4411 — Reinforcement Learning and Agentic AI

University of Oslo, Department of Physics

2026

Reinforcement learning gives a formal language for interaction: an agent observes a situation, selects an action, receives feedback, and improves behavior over time.

Agentic AI extends this language to systems built around foundation models that can retrieve information, call tools, write and execute code, coordinate with other models, and interact with humans or scientific infrastructure.

The purpose of this lecture is to identify the conceptual continuity between the two settings, not to treat every agent architecture in detail.

Learning goals

After this lecture, you should be able to explain how core RL notions reappear in agentic AI:

state, observation, action, feedback, policy, value, evaluation.

You should also be able to describe why scientific applications motivate tool use, retrieval, code execution, multi-agent organization, and human oversight.

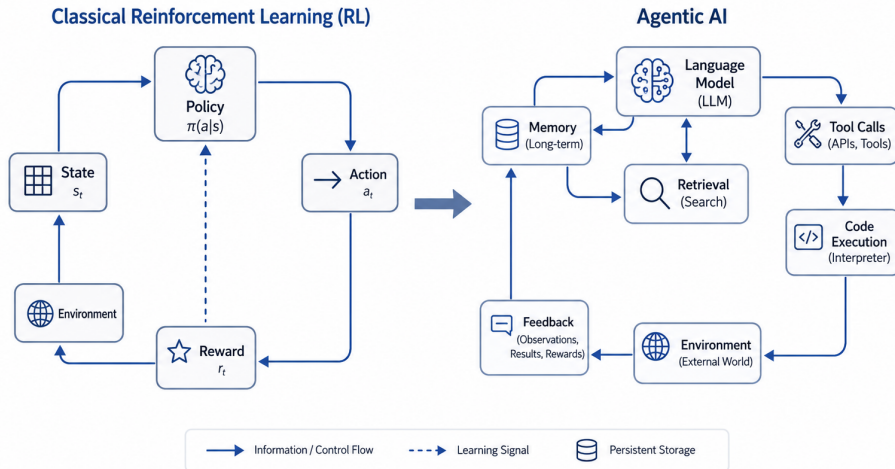
The final goal is to see agentic AI as a structured computational system, not simply as a more verbose language model.

Theme	Role in the lecture
RL foundations	Sequential interaction and feedback
Foundation models	From prediction to reasoning interfaces
Agentic systems	Tools, memory, retrieval, and execution
Science applications	Closed-loop discovery and research workflows
Evaluation	Verification, safety, and trust boundaries

Later lectures can treat each component in depth. Here we emphasize the common structure.

From an RL loop to an agentic system

From Reinforcement Learning to Agentic AI



Part I: Reinforcement learning foundations

Sequential interaction as the common formal language

The agent-environment interface

Reinforcement learning starts from an interaction protocol rather than a static training set.

$$S_t \longrightarrow A_t \longrightarrow (R_{t+1}, S_{t+1}).$$

The state can represent a physical configuration, a simulator state, a belief state, a code repository, or a conversation context.

The action can be a motor command, a control pulse, a query, a function call, or an edit to a file.

An MDP describes a decision process by

$$\mathcal{M} = (\mathcal{S}, \mathcal{A}, p, r, \gamma).$$

The transition kernel

$$p(s', r \mid s, a)$$

encodes how the environment responds to actions.

The formalism does not require the environment to be simple. It only requires that the state representation contains the information needed for prediction at the chosen level of abstraction.

A policy defines the agent's action rule:

$$\pi(a \mid s) = \Pr(A_t = a \mid S_t = s).$$

In tabular RL, this may be a table. In deep RL, it may be a neural network. In agentic AI, it may be a reasoning model combined with prompts, tools, memory, retrieval, and control logic.

The common question is still: given the information available now, what action should be selected next?

RL evaluates actions by their long-term consequences.

$$G_t = \sum_{k=0}^{T-t-1} \gamma^k R_{t+k+1}, \quad v_{\pi}(s) = \mathbb{E}_{\pi}[G_t \mid S_t = s].$$

The value function is a predictive object. It asks not only what happened immediately, but what a state makes possible under a policy.

This distinction is central when actions include information gathering, tool use, or experiment selection.

The action-value function resolves value over actions:

$$q_{\pi}(s, a) = \mathbb{E}_{\pi}[G_t \mid S_t = s, A_t = a].$$

A greedy decision rule can be written as

$$a^*(s) \in \arg \max_a q(s, a).$$

In agentic systems, the same idea appears when a model decides whether to retrieve, calculate, call a tool, ask the user, write code, or stop.

Sequential tasks rarely give immediate evidence about which earlier action mattered.

early action $\longrightarrow \dots \longrightarrow$ delayed outcome.

Credit assignment is the problem of attributing success or failure to earlier decisions. It is present in games, robotics, quantum control, and agentic workflows.

For tool-using agents, the question becomes: which retrieval, code execution, hypothesis, or intermediate decision contributed to the final result?

Exploration is not merely random behavior. It is the acquisition of information that may improve later decisions.

$$\text{action value} = \text{reward value} + \text{information value}.$$

This is especially relevant in scientific settings, where an experiment or simulation may be valuable because it reduces uncertainty, even when it does not immediately optimize the objective.

Model-free and model-based viewpoints

Model-free

The agent learns behavior or value directly from interaction data. The transition model is not used explicitly for planning.

Model-based

The agent uses a model of the environment to predict consequences, evaluate counterfactuals, or plan before acting.

Agentic AI often combines both viewpoints: a language model may act directly, while tools, simulators, retrieval systems, and learned models provide structured prediction.

In many applications the agent does not observe the full state:

$$O_t \neq S_t.$$

The policy is then conditioned on observations or histories:

$$\pi(a \mid O_1, A_1, \dots, O_t).$$

This is familiar from quantum measurement, robotics, and scientific agents that only see retrieved documents, logs, plots, intermediate files, and user feedback.

Deep RL replaces tables by parametric approximators:

$$q(s, a) \approx q_{\theta}(s, a), \quad \pi(a \mid s) \approx \pi_{\theta}(a \mid s).$$

The same conceptual move occurs when large language models provide a high-capacity conditional distribution over text, actions, plans, and tool calls.

The difference is that the action representation is no longer a small fixed index set. It can include structured language and executable operations.

Part II: From foundation models to agents

Language models as components in interactive systems

A foundation model is a general-purpose model trained on broad data and adapted to many downstream tasks.

For language models, the basic predictive object is

$$p_{\theta}(x_{t+1} \mid x_{\leq t}).$$

This conditional distribution is not yet an agent. It becomes agentic when embedded in an interaction loop with goals, tools, memory, and feedback.

Reasoning-oriented models are trained or prompted to produce intermediate computations, plans, or explanations before final answers.

In a scientific setting, the relevant behavior is not only linguistic fluency. It is the ability to coordinate symbolic reasoning, domain knowledge, numerical tools, and evidence.

This motivates architectures where the model is not the whole system, but the central controller in a larger computational workflow.

From model output to system action

Model output	System interpretation
Text answer	Communication with user
Search query	Retrieval action
Function call	Tool invocation
Python code	Executable computation
Plan or critique	Control signal for later steps

Agentic AI begins when outputs are interpreted as actions with consequences.

A formal analogy

RL notation can be used as a diagnostic lens:

O_t = context, files, observations, tool results,

A_t = message, query, tool call, code execution, edit,

R_{t+1} = task success, human feedback, tests, evaluator score.

This analogy is not exact training equivalence. It is a way to analyze interaction, feedback, and control.

ReAct-style reasoning and acting

A central idea in tool-using language agents is to interleave reasoning traces and environment actions.

reason $\rightarrow$ act $\rightarrow$ observe $\rightarrow$ reason.

The motivation is clear: the model can use intermediate observations to correct or refine subsequent steps.

This is close in spirit to closed-loop control, but the actions may be search, retrieval, calculation, code execution, or API calls.

Motivated by ReAct: reasoning and acting in language models.

Recent agentic systems are often better described as compound AI systems than as single models.

model + retrieval + tools + control logic + evaluation.

This framing is important for science. Scientific tasks rarely require only text generation. They require access to data, software, models, simulations, and measurement systems.

Motivated by the Berkeley AI Research discussion of compound AI systems.

Retrieval-augmented generation adds an external memory mechanism:

query $\rightarrow$ retrieve documents $\rightarrow$ condition response.

The purpose is not simply to add facts. Retrieval creates a controlled interface between the model and a knowledge source.

For scientific use, this improves traceability only when sources are relevant, cited, and checked.

Tool use changes the role of the model from generator to controller.

LLM $\rightarrow$ structured call $\rightarrow$ external system $\rightarrow$ observation.

The external system may be a calculator, search engine, database, simulator, theorem prover, code interpreter, microscope, robot, or HPC workflow.

The scientific value depends on the reliability of the tool interface and the validation of outputs.

Agent memory can be short-term, persistent, or retrieved from external stores.

$$M_t = f(M_{t-1}, O_t, A_t, \text{feedback}).$$

Memory is useful when tasks require continuity: tracking hypotheses, prior experiments, user constraints, code state, or previous failures.

It also creates risks. Stored information can become stale, irrelevant, or misleading if not curated.

Human oversight is not an afterthought. In scientific work, the human defines objectives, judges plausibility, interprets results, and decides when evidence is sufficient.

An agent can assist with search, coding, critique, experiment design, and workflow execution.

The scientist remains responsible for the research question, assumptions, validation, and conclusions.

Part III: Agentic architectures for science

Tools, retrieval, memory, and execution as scientific infrastructure

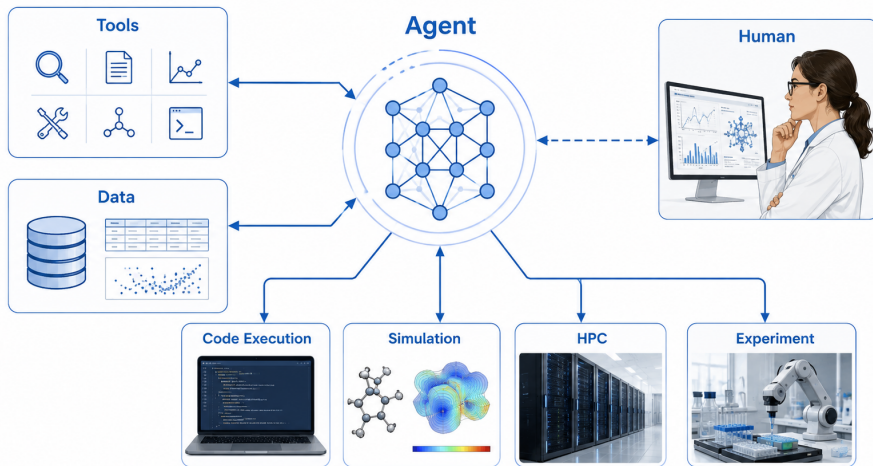
Scientific Discovery Platforms can be understood as AI-native systems that connect reasoning models to scientific resources.

reasoning model $\leftrightarrow$ data, codes, instruments, workflows, humans.

The goal is to support complete research processes: hypothesis generation, experimental design, computation, analysis, and verification.

Terminology aligned with the Agents for Science curriculum.

Agentic architecture for scientific work



A useful abstraction for agents is the cycle

sense $\rightarrow$ plan $\rightarrow$ act $\rightarrow$ learn.

In scientific systems, sensing may mean reading literature, inspecting data, observing experiments, or receiving tool outputs.

Acting may mean querying a database, running code, launching a simulation, proposing a molecule, or asking a human for clarification.

Terminology aligned with Lecture 1 of the AI Agents for Science curriculum.

Agentic systems often operate on heterogeneous observations:

Observation type	Example
Text	articles, protocols, user instructions
Data	tables, spectra, time series
Code state	files, errors, test results
Simulation state	trajectories, fields, converged outputs
Experiment state	measurements, instrument logs

This heterogeneity is one reason scientific agents require tool-mediated interaction.

In classical RL, the action space is often defined in advance. In agentic AI, actions are structured and contextual.

$$A_t \in \{\text{retrieve, call tool, execute code, edit, ask, stop}\}.$$

Each action has preconditions, costs, and possible failure modes.

A central design problem is therefore action governance: which actions are permitted, logged, verified, and reversible?

Tool use requires more than allowing arbitrary text. A tool call should have a schema:

name, arguments, types, constraints, return format.

Formal interfaces reduce ambiguity and make execution auditable.

In scientific settings, schemas should also encode units, valid ranges, provenance, and expected uncertainty when possible.

Motivated by tool-calling and Model Context Protocol themes in the Agents for Science curriculum.

Code execution turns language into computation.

hypothesis $\rightarrow$ script $\rightarrow$ run $\rightarrow$ output $\rightarrow$ interpretation.

This is powerful because it allows agents to calculate, test, plot, optimize, and inspect intermediate results.

It is also risky. Generated code can silently implement the wrong equation, use inconsistent units, or pass superficial tests.

Many scientific questions require simulations that exceed a local notebook.

An agentic workflow may submit jobs, monitor logs, adapt parameters, retrieve outputs, and summarize results.

proposal → job submission → monitoring → analysis.

The challenge is not only automation. It is reliable orchestration under queue delays, failures, data movement, and resource constraints.

Closed-loop experimentation



A self-driving laboratory closes the loop between model, experiment, and decision-making.

design $\rightarrow$ execute experiment $\rightarrow$ measure $\rightarrow$ update model $\rightarrow$ next design.

Agentic AI enters when language models help coordinate literature, hypotheses, protocols, instrument actions, analysis scripts, and human review.

This setting is a direct descendant of RL thinking: sequential decisions under uncertainty with costly feedback.

Multi-agent organization

A single model can in principle perform many roles. In practice, scientific workflows often benefit from specialization.

Role	Typical function
Knowledge agent	retrieval and synthesis
Planning agent	workflow decomposition
Coding agent	implementation and execution
Critic agent	verification and error finding
Experiment agent	protocol and instrument interface

The central issue is coordination, not merely adding more models.

Part IV: Scientific use cases

Motivating studies and application patterns

Why science is a demanding domain

Scientific work combines symbolic reasoning, noisy data, expensive experiments, specialized software, and standards of evidence.

This makes science an attractive domain for agentic systems, but also a domain where fluent language is insufficient.

A scientific agent must be evaluated on validity, reproducibility, provenance, and the usefulness of its contribution to inquiry.

Agentic AI is often motivated by an accelerated scientific loop:

literature $\rightarrow$ hypothesis $\rightarrow$ model $\rightarrow$ experiment $\rightarrow$ revision.

The agentic contribution is not a single prediction. It is the coordination of information extraction, hypothesis generation, simulation, experimental design, and analysis.

This is where the sense-plan-act-learn abstraction becomes scientifically concrete.

Study pattern: antimicrobial peptide design

Antimicrobial peptide design illustrates the need for coordinated expertise: sequence search, structural prediction, toxicity constraints, literature evidence, and experimental validation.

A useful agentic decomposition is

knowledge $\rightarrow$ candidate generation $\rightarrow$ structure $\rightarrow$ evaluation.

This resembles a multi-stage decision process over candidate molecules, where intermediate evidence affects later actions.

Motivated by the peptide-design example in Lecture 1 of the AI Agents for Science curriculum.

Study pattern: Virtual Lab

A Virtual Lab style workflow organizes multiple role-specialized agents around a scientific objective.

Component	Scientific role
Principal investigator	objective and coordination
Domain experts	specialized hypotheses
Critic or reviewer	failure detection
Tool interface	computation and external resources
Human scientist	judgment and validation

The important concept is structured collaboration, not model plurality for its own sake.

Study pattern: AI Scientist

Automated scientific discovery systems often combine idea generation, implementation, experimentation, and review.

idea → code → experiment → paper draft → review.

This is an appealing long-horizon task because it resembles research practice.

It is also difficult to evaluate because novelty, correctness, and scientific value are not reducible to a single benchmark score.

Motivated by AI Scientist and AI Scientist-v2 style systems.

Study pattern: self-driving laboratories

Self-driving laboratories combine autonomous decision-making with physical experimentation.

model uncertainty $\rightarrow$ experiment choice $\rightarrow$ measurement $\rightarrow$ model update.

This is one of the clearest scientific descendants of RL and Bayesian experimental design.

The agent must reason about cost, uncertainty, constraints, and the value of information.

Use case: quantum technology

Quantum technology offers natural agentic workflows: literature search, circuit construction, pulse optimization, noise characterization, decoder evaluation, and hardware calibration.

The environment may be a simulator, a backend, a laboratory control stack, or a repository of experimental data.

RL contributes the language of sequential control. Agentic AI contributes interfaces to tools, code, documentation, and human collaboration.

Use case: molecular dynamics

Molecular dynamics motivates agents that coordinate simulation, force-field selection, trajectory analysis, rare-event sampling, and active learning.

The decision problem is often not a single prediction, but where to spend the next expensive computation.

current data → next simulation → new evidence → updated model.

Use case: robotics and laboratories

Robotics makes the agent-environment relation physically explicit.

The agent observes sensors, selects actions, and receives feedback from the world. In laboratories, the same structure appears in sample handling, measurement protocols, and adaptive experimentation.

Agentic AI adds a language-mediated layer for planning protocols, interpreting logs, generating code, and coordinating between human intent and robotic execution.

Part V: Evaluation, trust, and the role of RL

Why interaction requires verification

Evaluation must match the task

For agents, evaluation cannot be limited to whether a final text answer sounds plausible.

A meaningful evaluation should examine task completion, tool correctness, robustness, provenance, cost, reproducibility, and behavior under perturbation.

This is why benchmark design for agents is difficult. The object being evaluated is a system interacting over time, not only a static model output.

Motivated by MLE-bench and AI Agents That Matter discussions of agent evaluation.

In software-oriented agents, tests provide an important feedback signal.

code edit $\rightarrow$ test suite $\rightarrow$ pass/fail observation.

The analogy with RL is direct: tests act as environmental feedback, although they are not necessarily a complete reward function.

A system that optimizes for passing tests may still be wrong if the tests do not encode the scientific requirement.

AI systems can increase productivity while weakening understanding if researchers accept fluent outputs without sufficient scrutiny.

The relevant risk is epistemic: a system may produce plausible code, explanations, citations, or experimental plans that create confidence without adequate evidence.

For scientific agents, the remedy is not rejection of automation. It is disciplined verification, explicit assumptions, provenance, and human judgment.

Motivated by Messeri and Crockett, *Artificial intelligence and illusions of understanding in scientific research*.

Failure modes

Failure mode	Scientific consequence
Hallucinated evidence	unsupported claims
Tool misuse	incorrect computation or action
Weak retrieval	incomplete literature basis
Benchmark overfitting	misleading performance estimates
Poor provenance	irreproducible result
Uncontrolled autonomy	unsafe or invalid workflow

The main design question is how to make these failures observable and correctable.

Why RL thinking remains useful

RL does not provide a complete theory of agentic AI. It does provide durable concepts for analyzing it.

interaction, feedback, delayed consequences, exploration, policy evaluation.

These concepts help formulate questions about agents: what information is observed, which actions are allowed, what feedback is reliable, and how performance is evaluated over time?

From RL to agentic AI

The transition can be summarized as an expansion of the agent interface.

Concept	RL	Agentic AI
Observation	state or features	context, files, data, tools
Action	control command	tool call, code, retrieval, text
Feedback	reward	tests, users, metrics, observations
Policy	learned mapping	model plus system design
Environment	simulator or world	software, humans, labs, HPC

Agentic AI is best viewed as a system-level development: foundation models are connected to memory, retrieval, tools, execution environments, and human collaborators.

For science, the motivating question is not whether agents can produce fluent text. It is whether they can help formulate, execute, evaluate, and document reliable research workflows.

The RL perspective remains valuable because it insists on interaction, feedback, and evaluation under uncertainty.

Selected sources and motivating studies

Theme	Representative source or study
Agents for science	AI Agents for Science curriculum and Lecture 1
Agent definitions	Cheng et al., LLM-based intelligent agents
Compound systems	Berkeley AI Research, Compound AI Systems
Reasoning and acting	ReAct
Retrieval	RAG and vector databases
Multi-agent systems	AutoGen, LangGraph, AIOS
Scientific workflows	Virtual Lab, self-driving laboratories, AI Scientist
Evaluation	MLE-bench, AI Agents That Matter
Epistemic caution	Messeri and Crockett, Nature